Project Ditto v2 — Session-by-Session Build Plan

Detailed handoff for Claude Code. Sessions are ordered, each produces specific deliverables, and gates interrupt progression if criteria fail.

Build plan v1.0 · paired with Ditto v2 spec v1.1 · 7 weeks total · ~$130-160 budget

Preamble: Read This First

Claude Code: before starting ANY session, read these four documents in order:

1. Ditto v2 spec v1.1 (the pre-registration document)
2. Ditto v1 repo's CLAUDE.md (to understand the v1 codebase this work extends)
3. Ditto v1 repo's `src/translation.py` (the frozen T-battle — your template for T-code structure)
4. This build plan

The v1 codebase at `github.com/safiqsindha/Project-Ditto` is the foundation. Do not rewrite modules that v1 already has unless the change is required by this plan. v1's parser, renderer skeleton, scorer, shuffler, and filter are reusable with source-specific adaptations.

Project structure

```
project-ditto-v2/
├── CLAUDE.md                    (architecture doc, copied + updated from v1)
├── SPEC.md                      (copy of v1.1 spec, frozen)
├── BUILD_PLAN.md                (this document in markdown)
├── data/
│   ├── terminal_bench/          (primary source raw trajectories)
│   ├── swe_bench_verified/      (secondary source raw trajectories)
│   └── human_sessions/          (validation source)
├── chains/
│   ├── real/tb/                 (Terminal-Bench real chains)
│   ├── real/swe/                (SWE-bench real chains)
│   ├── real/human/              (human-session validation)
│   ├── shuffled/tb/
│   ├── shuffled/swe/
│   └── shuffled/human/
├── src/
│   ├── parser_tb.py             (new: Terminal-Bench trajectory parser)
│   ├── parser_swe.py            (new: SWE-bench trajectory parser)
│   ├── parser_human.py          (new: human-session parser)
│   ├── translation.py           (new: T-code, replaces v1's T-battle)
│   ├── renderer.py              (adapted from v1, with expanded leakage list)
│   ├── aggregation.py           (new: source-specific event aggregation)
│   ├── filter.py                (copied from v1, unchanged)
│   ├── shuffler.py              (copied from v1, unchanged)
│   ├── observability.py         (adapted: code-region hiding instead of opponent hiding)
│   ├── reference.py             (adapted from v1)
│   ├── normalize.py             (copied from v1)
│   ├── prompt_builder.py        (new version: prompt_v2.0-code)
│   ├── runner.py                (copied from v1 with source-selection flag)
│   └── scorer.py                (copied from v1, unchanged)
├── scripts/
│   ├── acquire_tb.py            (new)
│   ├── acquire_swe.py           (new)
│   ├── acquire_human.py         (new)
│   ├── build_chains.py          (adapted: accepts --source flag)
│   ├── build_reference.py       (adapted: builds per-source distributions)
│   └── run_evaluation.py        (adapted: iterates over sources)
├── tests/
│   ├── test_parser_tb.py
```

```
│   ├── test_parser_swe.py
│   ├── test_translation.py      (new test suite — T-code is not T-battle)
│   ├── test_aggregation.py
│   ├── test_leakage.py          (programming-vocabulary leakage tests)
│   └── test_shuffler.py         (copied from v1)
└── results/
    ├── raw/
    ├── blinded/
    └── scored.json
```

Shared conventions

- Python 3.11+. Same versions as v1's requirements.txt plus whatever Terminal-Bench or SWE-bench harness requires.
- Commit after each session with a clear message: `Session N: [deliverable]`.
- All scripts must run successfully from the repo root.
- All file paths in scripts should use `pathlib.Path`.
- Use `argparse` for CLI flags. No hard-coded paths except for fallback defaults.
- Every new module gets a module-level docstring explaining its role in the pipeline.
- When in doubt, copy patterns from v1's codebase rather than inventing new ones.

Session 1 — Repo scaffolding + environment setup

Session 1 · Week 1 · Est. 3–4 hours of work

Inputs: Ditto v1 repo, v2 spec v1.1

Outputs: Fresh repo skeleton, env setup, v1 reuse strategy documented

Dependencies: None

Tasks

1. Create new repo `project-ditto-v2`. Do not fork v1 — start fresh but copy specific files.
2. Copy these files from v1 unchanged: `src/filter.py`, `src/shuffler.py`, `src/normalize.py`, `src/scorer.py`, `tests/test_shuffler.py`. These work on abstract constraint objects and don't need domain adaptation.
3. Copy these as starting points that will be adapted: `src/renderer.py`, `src/observability.py`, `src/reference.py`, `src/runner.py`, `src/prompt_builder.py`.
4. Do NOT copy: `src/translation.py`, `src/parser.py`, `scripts/*`, `tests/test_translation.py`. These are domain-specific and need full rewrites.
5. Create `pyproject.toml` and `requirements.txt`. Start from v1's and add: `inspect-ai` (for Terminal-Bench harness compatibility), `harbor-framework` if available, otherwise Terminal-Bench's `terminal-bench` Python package.
6. Create `.env.example` with `ANTHROPIC_API_KEY` and (if needed) a HuggingFace token for dataset access.
7. Create the full directory structure shown in the Preamble.
8. Copy the v1.1 spec PDF into the repo as `SPEC.pdf`. Create a `SPEC.md` that reproduces the key pre-registration sections (thresholds, gates, methodology commitments). Commit with message **"Pre-registration commit — thresholds and methodology frozen"**.
9. Tag the commit: `git tag -a v2-preregistration -m "Pre-registration frozen per spec v1.1" && git push origin v2-preregistration`.
10. Write the new `CLAUDE.md` — copy v1's and update the architecture, pipeline diagram, and file listing to match v2's two-source design.

Deliverables

- Repo initialized with full directory structure
- v1 files copied in correctly categorized as "unchanged", "to adapt", or "rewrite"
- Pre-registration committed and tagged before any domain work begins
- CLAUDE.md accurately describes v2 architecture

> **Gate 1 check:** Can `pytest tests/test_shuffler.py` run and pass against the copied shuffler? If not, the copy has broken imports and must be fixed before Session 2.

Session 2 — Terminal-Bench trajectory acquisition

Session 2 · Week 1 · Est. 4–6 hours

Inputs: Terminal-Bench 2.0 public evaluation data

Outputs: ~500 parsed Terminal-Bench trajectories

Dependencies: Session 1 complete

Tasks

1. Research Terminal-Bench 2.0 trajectory data availability. Check:
   - Harbor framework GitHub: `github.com/laude-institute/terminal-bench`
   - Terminal-Bench experiments repo: `github.com/laude-institute/terminal-bench-experiments`
   - Published model evaluation runs (Terminus-2, Claude Code, Codex CLI)
   - Any associated HuggingFace datasets
2. Document what trajectory data is publicly available and in what format. Write a short `data/terminal_bench/SOURCE.md` file documenting exactly where data came from, when it was downloaded, and the schema.
3. Write `scripts/acquire_tb.py`:
   - Accepts `--source` (local path or URL), `--out`, `--target N`
   - Parses raw Terminal-Bench trajectory format into canonical JSONL
   - Each record: `{task_id, model, agent, outcome (pass/fail), trajectory: [events]}`
   - Events should capture: bash commands, command outputs, error messages, final patches
   - Filter: trajectories with ≥ 15 events, excluding early-terminated runs
4. Write `src/parser_tb.py`:
   - Function `parse_tb_trajectory(raw: dict) -> TrajectoryLog`
   - `TrajectoryLog` is a new dataclass analogous to v1's `BattleLog`
   - Emit a list of `TrajectoryEvent` objects — analogous to v1's `BattleEvent`
   - Event types: `bash_call`, `file_read`, `file_edit`, `test_run`, `error_reveal`, `context_update`, `task_transition`
5. Write basic `tests/test_parser_tb.py` with at least 3 fixtures from real data.
6. Run `scripts/acquire_tb.py --target 500`. Report the acquired count.

Deliverables

- `data/terminal_bench/` contains 400+ parsed trajectories (Gate 1 minimum)
- `SOURCE.md` documents data provenance
- Parser tested against real examples

> **Gate 2a check:** ≥ 400 usable Terminal-Bench trajectories after filtering. If below 400: try additional model/agent pairs, relax filter criteria with user approval, or escalate to human review for alternative data source selection before proceeding.

> **Safiq (human) review point after Session 2:** Before Session 3, manually inspect 10 parsed trajectories. Confirm they contain the event types you expected. Confirm the parser is not dropping events silently. This takes ~20 minutes and catches schema bugs that unit tests miss.

Session 3 — SWE-bench Verified trajectory acquisition

Session 3 · Week 1–2 · Est. 4–6 hours

Inputs: SWE-bench Verified + published agent trajectories

Outputs: ~500 parsed SWE-bench trajectories

Dependencies: Session 2 complete

Tasks

1. Research SWE-bench Verified trajectory availability. Check:
   - SWE-bench main repo: `github.com/princeton-nlp/SWE-bench`
   - SWE-agent repo: `github.com/princeton-nlp/SWE-agent`
   - mini-SWE-agent published trajectories (used for Claude model evaluations)
   - Vals.ai and other leaderboard-associated public trajectory dumps
2. Document source in `data/swe_bench_verified/SOURCE.md`.
3. Write `scripts/acquire_swe.py` — same interface as `acquire_tb.py`, parses SWE-bench trajectory format.
4. Write `src/parser_swe.py`:
   - Parses SWE-agent / mini-SWE-agent trajectory format
   - Produces `TrajectoryLog` with the same `TrajectoryEvent` types
   - Handles SWE-specific idiosyncrasies: file-read bursts, long error stacks, git diff events
5. Write `tests/test_parser_swe.py` with 3+ fixtures.
6. Run `scripts/acquire_swe.py --target 500`. Report count.

Deliverables

- `data/swe_bench_verified/` contains 400+ parsed trajectories
- Both parsers produce the same `TrajectoryEvent` type vocabulary (critical for T-code uniformity)

Gate 2b check: ≥ 400 usable SWE-bench Verified trajectories. Same failure response as Gate 2a.

Safiq review point after Session 3: Inspect 10 SWE-bench trajectories. These are typically much longer (100+ events) than Terminal-Bench trajectories (20-50 events). Confirm this matches expectations, because it drives the aggregation design in Session 5.

Session 4 — Human debugging session acquisition (validation set)

Session 4 · Week 2 · Est. 3–5 hours

Inputs: Public human debugging data sources	**Outputs:** ~100 parsed human trajectories	**Dependencies:** Session 3 complete

Tasks

1. Identify public human-debugging data. Options in priority order:
   - Public Twitch coding streams with timestamp-aligned terminal/IDE actions
   - Stack Overflow "walk me through debugging" answer threads where the human described sequential steps
   - GitHub issue threads where a developer described step-by-step debugging in comments
   - Live-coding YouTube videos with terminal screencapture (requires transcription)
2. Target 100 trajectories, not 500. This is a validation set, not a primary source.
3. Document in `data/human_sessions/SOURCE.md`. Include: source URL, date accessed, any consent/license considerations.
4. Write `scripts/acquire_human.py` and `src/parser_human.py`. The parser may need to be more lenient about event granularity — human descriptions are less structured than agent trajectories.
5. If achieving 100 high-quality human trajectories proves too hard in one session, stop at whatever count you have (minimum 30) and flag it to human review.

Deliverables

- `data/human_sessions/` with ≥ 30 parsed human trajectories
- Source provenance documented

> **If data acquisition for this session is hard or legally ambiguous:** Do not scrape copyrighted material. Do not screen-scrape Twitch without permission. Do not use YouTube transcripts that may violate ToS. Stop and escalate to human review rather than getting data through questionable means. The validation set is nice-to-have; the two primary sources are what the experiment depends on.

> **Safiq review point after Session 4:** Review the data sources Claude Code chose. Confirm they're licensed for research use or are in sufficiently public domain. If not, have Claude Code find alternatives. This step matters for external publication later.

Session 5 — Aggregation rules module

Session 5 · Week 2–3 · Est. 4–6 hours

Inputs: Parsed trajectories from Sessions 2-4

Outputs: Source-specific aggregation logic

Dependencies: Sessions 2-4 complete

Tasks

1. Implement `src/aggregation.py` with source-specific aggregation rules per spec §3.4:
   - `aggregate_tb_events(events) -> events'`: mostly pass-through, filter no-op commands, collapse consecutive identical bash calls
   - `aggregate_swe_events(events) -> events'`: collapse file-read bursts, truncate error stacks to first-line, compress retry loops
   - `aggregate_human_events(events) -> events'`: lenient, since human events are already higher-granularity
2. Each aggregation rule must be pure (deterministic, no external state).
3. Each rule produces a log of what it aggregated (for inspection).
4. Write `tests/test_aggregation.py` with fixtures showing: (a) inputs, (b) expected aggregated outputs, (c) rationale for each rule.
5. Run aggregation on all parsed trajectories. Confirm output event counts are within target ranges (Terminal-Bench ~15-50 events per chain, SWE-bench ~30-80 after aggregation).

Deliverables

- `src/aggregation.py` with three source-specific rules
- Test coverage on aggregation behavior
- Post-aggregation event count distributions within target ranges

Critical: Aggregation rules are part of T-code and will be frozen at the same time. Do not tune aggregation rules after looking at scored results — that would invalidate pre-registration. Commit aggregation rules before Session 6.

Safiq review point after Session 5: Spot-check aggregation on 5 trajectories per source. Confirm aggregated events look like a sensible summary, not a broken compression. This is a research-judgment step, not an engineering step.

Session 6 — T-code translation function (the research artifact)

Session 6 · Week 3 · Est. 8–12 hours (the longest session)

Inputs: Aggregated trajectories, v1's T-battle as reference

Outputs: Working T-code (not yet frozen)

Dependencies: Session 5 complete

Tasks

1. Implement `src/translation.py`. Structure mirrors v1's `translation.py`:
   - Import the six `Constraint` dataclasses from v1's translation.py (they should be identical)
   - `TranslationContext` class tracking per-trajectory state (file label mapping, command label mapping, phase, resource counters)
   - `translate_event(event, ctx) -> list[Constraint]` dispatcher
   - Individual translator functions per event type (per the mapping table in spec §3.1)
   - `translate_trajectory(events, source) -> list[Constraint]` top-level function
2. Implement each event-to-constraint mapping:
   - **bash_call:** `ToolAvailability` with tool=command_class
   - **file_read:** `InformationState` with `observable_added=[file_label]`
   - **file_edit (create/delete):** `ToolAvailability`
   - **file_edit (modify):** `SubGoalTransition`
   - **test_run pass:** `SubGoalTransition` to phase=validated
   - **test_run fail:** `SubGoalTransition` + `InformationState`
   - **error_reveal:** `InformationState` with error class
   - **context_update:** `ResourceBudget` for context_window
   - **retry pattern:** `ResourceBudget` for patience_budget
   - **module discovery:** `CoordinationDependency`
   - **task_transition:** `SubGoalTransition` with phase label
3. All labels must be abstract: `file_A..file_Z`, `command_1..command_N`, `error_class_A..Z`. No domain vocabulary.
4. Implement `_get_file_label()`, `_get_command_label()`, `_get_error_label()` helpers (mirror v1's `_get_unit_label()` pattern).
5. Overflow handling: if a trajectory uses > 26 distinct files, the 27th and beyond get `file_OVERFLOW_N`. Log a warning. Do not silently reuse labels (this was a v1 bug).
6. Implement `tests/test_translation.py` with at least 30 unit tests covering all mapping cases. Each test: construct minimal fixture event, call `translate_event`, assert constraint type and field values.
7. Run `translate_trajectory` on 10 real trajectories from each source. Manually inspect outputs. Adjust T-code if any event produces obviously wrong constraints.

Deliverables

- Complete T-code in `src/translation.py`
- 30+ passing unit tests
- Manual inspection confirms translation quality on sample trajectories

> **Do not freeze T-code yet.** Session 7's renderer and leakage check may reveal issues that require T-code adjustments. Freezing happens in Session 8.

> **Safiq review point after Session 6 (CRITICAL):** This is the most important review in the whole build. Spend 1-2 hours reading 20 rendered trajectories (10 per source). Ask: "Does this look like a real programming session reduced to

> abstract structure? Does it capture the decisions, constraints, and state changes, or does it feel mechanical?" If T-code feels too mechanical, redesign before Session 7. If it feels right, proceed. This is where research taste matters most.

Session 7 — Renderer + vocabulary leakage check

Session 7 · Week 3–4 · Est. 4–6 hours

Inputs: T-code output	**Outputs:** Rendered chains, validated leakage check	**Dependencies:** Session 6 complete

Tasks

1. Adapt `src/renderer.py` from v1:
   - Per-constraint-type rendering functions remain largely unchanged from v1
   - Update any v1-specific wording (e.g., battle-specific framing in headers)
   - Add a new `render_trajectory_chain()` function if needed, or reuse `render_chain()` directly
2. Build the programming-vocabulary leakage list. Include:
   - Common Python packages: django, flask, numpy, pandas, scipy, sklearn, matplotlib, torch, tensorflow, requests, fastapi, pydantic, pytest, sqlalchemy, etc. (aim for 100+ names)
   - Python language keywords: class, def, return, import, async, await, yield, lambda, etc.
   - Common bash commands: grep, find, sed, awk, ls, cat, git, make, pip, npm, docker, curl, ssh, etc.
   - Error type names: TypeError, ValueError, KeyError, AttributeError, ImportError, etc.
   - File extensions: .py, .js, .ts, .md, .yaml, .json (in context, e.g., "file.py" shouldn't appear)
3. Ship this vocabulary as a default list in the module. Make `check_programming_leakage()` use this by default — callers cannot skip the check.
4. Modify `render_chain()` to automatically invoke the leakage check and raise `ValueError` on any hit (not just return a list).
5. Write `tests/test_leakage.py`:
   - Test that a constraint with a leaking token raises
   - Test that clean rendered output passes
   - Test that case-insensitive whole-word matching works correctly (no false positives on substrings)
   - Test the default vocabulary list loads correctly
6. Run rendering on 20 sample chains per source. Run leakage check. Fix any leaks by either (a) improving T-code's abstraction or (b) adding more vocab to the leakage list.

Deliverables

- Working renderer that automatically runs leakage check
- Default programming-vocabulary leakage list (100+ entries)
- Tests passing on real rendered samples

> **Gate 3 check:** Render 20 chains per source (40 total). Leakage check must pass on all 40. If any chain leaks, stop and fix T-code or expand vocab list before Session 8. Do not freeze T-code until this gate passes.

Session 8 — Freeze T-code, build observability layer

Session 8 · Week 4 · Est. 4–5 hours

Inputs: T-code + renderer from Session 7	**Outputs:** Frozen T-code, working observability layer	**Dependencies:** Sessions 6-7 complete, Gate 3 passed

Tasks

1. Run final validation:
   - All unit tests pass: `pytest tests/`
   - Leakage check passes on 50+ rendered chains per source
   - Manual spot-check on 10 chains per source
2. **Freeze T-code:** `git tag -a T-code-v1.0-frozen -m "T-code frozen per spec §3. No further modifications to translation.py, aggregation.py, or the default leakage vocabulary without a new pre-registration." && git push origin T-code-v1.0-frozen`.
3. Adapt `src/observability.py` from v1:
   - v1 hid opponent units until revealed via switch/move events
   - v2 should hide file regions, module names, and error types until referenced by a read/execution event
   - The abstraction is: "information state constraints should only reference entities that have been `observable_added` in a prior InformationState event"
   - Apply same timing logic as v1's asymmetric reveal
4. Write simple tests for observability: given a trajectory with known file-read-then-reference pattern, confirm the pre-reveal reference is hidden.

Deliverables

- `T-code-v1.0-frozen` git tag exists on the remote
- Working observability layer
- All tests passing

> **Post-freeze discipline:** After this session, any modification to `src/translation.py`, `src/aggregation.py`, or the leakage vocabulary list is a pre-registration violation. Changes after this point must be documented explicitly as a new T-code version, and the experiment would need to be re-run. This is how research integrity works — the research artifact is the thing that cannot be tuned to produce the desired result.

> **Safiq review point after Session 8:** Verify the tag exists on GitHub. Run `git diff T-code-v1.0-frozen -- src/translation.py src/aggregation.py src/renderer.py` and confirm it returns empty. If not, the tag was created at the wrong commit — fix before Session 9.

Session 9 — Chain construction pipeline (both sources)

Session 9 · Week 5 · Est. 5–7 hours

Inputs: Frozen T-code, parsed trajectories from all three sources

Outputs: ~1100 real chains + ~3300 shuffled chains

Dependencies: Session 8 complete

Tasks

1. Adapt `scripts/build_chains.py` from v1:
   - Accept `--source` flag: `tb`, `swe`, or `human`
   - Accept `--data`, `--out-real`, `--out-shuffled`, `--seeds`
   - Pipeline: parse → aggregate → translate → observability → window → filter → shuffle → save
   - Reuse v1's `_find_valid_window()` logic adapted for trajectory-based chains
   - Remove v1's `action_at_step` field entirely — it was dead code in v1's scorer
2. Run chain construction for each source:
   - `python scripts/build_chains.py --source tb --data data/terminal_bench/ --out-real chains/real/tb/ --out-shuffled chains/shuffled/tb/`
   - Same for `swe` and `human`
3. Target counts:
   - Terminal-Bench: 500 real, 1500 shuffled (3 seeds)
   - SWE-bench: 500 real, 1500 shuffled
   - Human: ≥ 30 real, ≥ 90 shuffled
4. Log diagnostic stats per source: length histogram, constraint type frequencies, number of trajectories rejected by validity filter and why.

Deliverables

- Chain files in `chains/real/<source>/` and `chains/shuffled/<source>/`
- Diagnostic summary per source

> **Intermediate check (not a formal gate):** If any source produces fewer than 300 valid real chains, stop and escalate. The statistical power of the pre-registered tests assumes ~500 chains per source; dropping below 300 changes the interpretation of non-significant results.

Session 10 — Reference distribution per source

Session 10 · Week 5 · Est. 4–6 hours

Inputs: Parsed trajectories + chain files

Outputs: Three reference distributions (one per source)

Dependencies: Session 9 complete

Tasks

1. Adapt `src/reference.py` from v1:
   - StateSignature structure may need per-source adaptation
   - For Terminal-Bench: active_pair could be (current_task_phase, last_command_class); hp_brackets becomes (context_remaining_bracket, patience_bracket)
   - For SWE-bench: state signature needs to include more file-read context
   - Decide during implementation, document the choice in module docstring
2. Adapt `scripts/build_reference.py`:
   - Build one reference distribution per source
   - Source: `build_reference.py build-raw --source tb --raw data/terminal_bench/ --out data/reference_tb.pkl`
   - Same for swe and human
   - Apply full action-string normalization (carried over from v1's fix #1)
3. Implement coverage check: `build_reference.py check --dist data/reference_tb.pkl --chains chains/real/tb/ --target 0.9`. Target ≥ 90% non-max-backoff coverage (tighter than v1's 80% to address v1's reference-miss issue).
4. Run reference build and coverage check for all three sources.

Deliverables

- Three reference distribution pickles
- Coverage reports per source

> **Gate 4 check:** Each source's reference distribution achieves ≥ 90% non-max-backoff coverage on a held-out sample. If any source falls below, expand that source's reference corpus (if possible) or document the limitation before evaluation.

Session 11 — Prompt builder + runner adaptation

Session 11 · Week 5–6 · Est. 3–4 hours

Inputs: Chain files + reference distributions **Outputs:** Working evaluation runner **Dependencies:** Session 10 complete

Tasks

1. Adapt `src/prompt_builder.py`:
   - PROMPT_VERSION = "v2.0-code"
   - Prompt text should be generic — say "adaptive pipeline with changing constraints" rather than anything programming-specific
   - The prompt must work identically on Terminal-Bench and SWE-bench chains — no source-specific branching
2. Adapt `src/runner.py`:
   - Same MODELS dict: `haiku` → `claude-haiku-4-5-20251001`, `sonnet` → `claude-sonnet-4-6`
   - Add `--source` flag to select which source's chains to evaluate
   - Full temperature/seed matrix from day 1: T=0 with seed 42 (primary), T=0.5 with seeds 1337 and 7919 (variance study)
   - Use Messages Batches API for 50% cost reduction
   - Output to `results/raw/<source>/<model>_<condition>_<seed>_T<temp>.json`
   - Copy blinded versions to `results/blinded/` — stripped of model, seed, and condition metadata
3. Write `scripts/run_evaluation.py` that iterates over (source, model, condition, seed, temperature) tuples and calls runner for each.
4. Do a dry run on 5 chains per source — confirm prompts look right and action outputs parse.

Deliverables

- Working runner with source flag
- Dry-run validation on 15 total chains

> **Gate 5 check:** Dry-run evaluation produces parseable responses AND non-zero Layer 1 match rates on dry-run scoring. If scoring returns near-zero on any source, debug format alignment (action normalization function) before committing the full batch spend. This is the check that would have caught v1's action-format bug earlier.

Session 12 — Full evaluation run

Session 12 · Week 6 · Est. 8–24 hours elapsed (mostly waiting)

Inputs: Validated runner

Outputs: ~21k raw + blinded responses

Dependencies: Session 11 complete, Gate 5 passed

Tasks

1. Submit batches for the full evaluation matrix:
   - 3 sources × 2 models × 2 conditions (real/shuffled) × 3 seeds = 36 batches
   - Expected total requests: ~21,000
   - Expected total cost: $80–130 on Batches API
2. Record each batch ID in `results/BATCH_LOG.md` with timestamps.
3. Monitor batch completion. Expected latency: Haiku batches ~1-4 hours, Sonnet batches ~4-12 hours.
4. As batches complete, download responses and save raw + blinded versions.
5. If any batch fails partially (like v1's credit-exhausted issue), re-submit the failed requests in a retry batch. Record retry batch IDs.
6. Handle rate limits by queuing rather than retrying naively.

Deliverables

- Full result set: `results/raw/<source>/*.json` and `results/blinded/*.json`
- Complete batch ID log
- Record of any failed/retried batches

> **Do not score during this session.** Scoring happens in Session 13 in a separate environment. Mixing evaluation and scoring in the same session was the one methodological limitation of v1; v2 explicitly avoids this.

Session 13 — Scoring (SEPARATE SESSION)

Session 13 · Week 7 · Est. 4–6 hours

Inputs: Blinded results (model/seed/condition stripped) **Outputs:** Scored results + outcome tier **Dependencies:** Session 12 complete

> **This session must be a fresh Claude Code session** — not a continuation of Session 12. Load only: the blinded results directory, the reference distribution pickles, and the scorer code. Do not load chain source files, raw results, or batch logs. The scorer has no business knowing which results came from which model, seed, or condition.

Tasks

1. Copy `src/scorer.py` from v1 (unchanged logic — v1's scorer works correctly).
2. Run scoring: `python -m src.scorer --results results/blinded/ --dist-tb data/reference_tb.pkl --dist-swe data/reference_swe.pkl --dist-human data/reference_human.pkl --chains-real chains/real/ --chains-shuffled chains/shuffled/ --out results/scored.json`
   - Scorer uses reference distribution via `extract_state_signature()` + `lookup()`, NOT any action_at_step field
   - Applies normalization to both model responses and reference keys before comparison
   - Computes Layer 1 (top-3 match) and Layer 2 (legality × optimality composite)
   - Runs Welch's t-test for Layer 2, two-sample proportion test for Layer 1
   - Classifies outcome tier per pre-registered thresholds
3. Re-unify with model/seed metadata ONLY after all scoring is complete (this is the last step).
4. Compute all pre-registered supplementary analyses per spec §5.1:
   - Effect-size comparison to v1
   - Primary-vs-secondary dataset consistency
   - Agent-trajectory vs human-trajectory consistency
5. Produce `results/scored.json` with all metrics per source × model × condition × seed × temperature.
6. Do NOT write the final results document yet — that's Session 14 after human review.

Deliverables

- `results/scored.json` with full Layer 1/2/3 breakdown per source × model
- Console output showing outcome tiers per source × model
- No writeup yet — just the numbers

> **Safiq review point after Session 13 (CRITICAL):** Before anything is published or written up, review the scored output carefully. Look for: outcome tiers per source, effect-size comparison to v1, whether both programming sources cleared thresholds or only one. This determines what the writeup claims — do not let Claude Code decide the framing before you see the numbers.

Session 14 — Results writeup + repo finalization

Session 14 · Week 7 · Est. 4–6 hours

Inputs: Scored results + human review from Session 13

Outputs: RESULTS.md, CHANGELOG update, clean repo

Dependencies: Session 13 complete + Safiq review

Tasks

1. Write `RESULTS.md` following the structure of v1's RESULTS.md:
   - TL;DR up top — honest statement of outcome tier
   - Headline numbers per source
   - Layer 2 composite
   - Pre-registration checklist (did each criterion clear?)
   - Cross-source consistency analysis
   - Comparison to v1 effect sizes
   - Multi-seed variance study
   - Method summary referencing SPEC.md
   - Interpretation with clear scoping of what the result proves and doesn't
   - Limitations section (honest, don't bury them)
   - Reproducibility: batch IDs, git tags, file artifacts
   - Next steps
2. Update `CHANGELOG.md` with Session-by-session summary.
3. Add `RESULTS.md` link to README.
4. Ensure all artifacts are committed: spec, build plan, results, batch IDs.
5. Create final git tag: `v2-complete`.
6. Human-review the writeup for honest framing — do not let positive results be oversold, do not let null results be buried.

Deliverables

- Complete RESULTS.md committed to repo
- Updated CHANGELOG
- Final v2-complete git tag
- Repo ready for public sharing

> **Final human review before publication:** Read the writeup end-to-end. Check: does the TL;DR match the numbers? Are limitations prominent enough? Does the "what this does not prove" section prevent overclaiming? If anything is softened from what the data actually supports, fix before publishing.

Summary table

Session	Week	Est. hours	Gate	Human review?
1 — Scaffolding	1	3-4	Gate 1	No
2 — Terminal-Bench acquisition	1	4-6	Gate 2a	Yes
3 — SWE-bench acquisition	1-2	4-6	Gate 2b	Yes
4 — Human sessions	2	3-5	-	Yes

5 — Aggregation	2-3	4-6	-	Yes
6 — T-code (most important)	3	8-12	-	**Yes, critical**
7 — Renderer + leakage	3-4	4-6	Gate 3	No
8 — Freeze T-code	4	4-5	-	Yes
9 — Chain construction	5	5-7	-	No
10 — Reference distribution	5	4-6	Gate 4	No
11 — Runner setup	5-6	3-4	Gate 5	No
12 — Full evaluation	6	8-24	-	No
13 — Scoring (fresh session)	7	4-6	-	**Yes, critical**
14 — Writeup	7	4-6	-	Yes, final

Total estimated work: 62-103 hours across 7 weeks. Roughly 2× the v1 build because of the two-source design, the tighter methodology, and the more rigorous freezing discipline.

Things Claude Code should NOT do

- **Do not modify T-code (`src/translation.py`, `src/aggregation.py`, `src/renderer.py`, or the leakage vocabulary) after Session 8's freeze tag.** This is a pre-registration violation.
- **Do not skip gates.** If a gate fails, stop and escalate to human review. Do not rationalize a borderline gate as passing.
- **Do not run scoring in Session 12.** Scoring must happen in Session 13's fresh session.
- **Do not write RESULTS.md before human review of raw scored output.** The writeup framing depends on what the numbers actually say.
- **Do not use synthetic data anywhere.** If Session 2, 3, or 4 cannot acquire real data, stop and escalate. v1's synthetic-data path caused real issues; v2 does not have a synthetic fallback.
- **Do not scrape copyrighted or ToS-protected material.** Especially in Session 4 for human sessions.
- **Do not "improve" v1's frozen code** (filter, shuffler, normalize, scorer) without explicit human approval. These are reused deliberately for comparability with v1.
- **Do not skip pre-registration commit in Session 1.** The pre-registration must happen before any domain-specific work.
- **Do not cache or memoize batch results without the proper seed-in-key.** Different seeds must produce different cached entries.
- **Do not modify pre-registered thresholds to match observed results.** If the numbers don't clear the thresholds, report that honestly.

Session handoff protocol

At the end of each session, Claude Code should write a short `SESSION_LOG.md` entry including:

- Session number and date
- Tasks completed
- Gate status (if applicable)
- Deviations from plan (if any — include justification)
- Files created or modified
- What the next session should do first
- Any blockers or uncertainties for human review

The log is written as the last action of each session. Do not start a new session without reading the previous session's log entry.

Build plan v1.0 paired with Ditto v2 spec v1.1. Estimated 62-103 hours across 7 weeks. Sessions 2, 3, 4, 5, 6, 8, 13, and 14 have human review points. Gates 1-5 provide stop conditions if something goes wrong. The methodology is tighter than v1's; the freezing discipline is stricter; the handoff is more explicit. This is what "publication-quality replication experiment" looks like at the build-plan level.